

Lab 3: Architectural Justification

Gyaan S Shetty | PES1UG24CS178 | Problem Statement 49: Webhook Ingestion and Retry Mechanism Hub

Architecture Selection

We chose **Microservices Architecture** (event-driven, broker-backed) for the Webhook Ingestion and Retry Mechanism Hub.

The system is split into seven independently deployable components: Ingestion Gateway, HMAC Auth Service, Event Queue (Broker), Forwarding Service, Retry Scheduler, Dead Letter Queue and Event Store, with the Third-Party Provider and Internal Consumer as external systems.

Reason 1: Ingestion and delivery have very different load profiles

Webhook traffic is bursty. A provider outage recovery or a batch event can send thousands of calls in seconds, while downstream delivery is limited by how fast internal consumers respond. Putting the Event Queue between the Ingestion Gateway and the Forwarding Service lets the Gateway acknowledge the provider immediately and scale on request volume, while forwarding workers scale separately on queue depth. A Layered design would tie both to one deployment; Client-Server would make the single server the bottleneck.

Reason 2: Failures in one stage must not stop the others

The core requirement of the Hub is reliability under failure. If an internal consumer is down, the Forwarding Service fails its delivery, hands the event to the Retry Scheduler (exponential backoff) and, once attempts are exhausted, the event is routed to the Dead Letter Queue. Ingestion keeps running throughout. Because each concern is a separate service behind a defined interface (IRetrySchedule, IDeadLetter), a crash or redeploy of the retry logic does not drop incoming webhooks, and each piece can be replaced without touching the rest.

Security Advantage

Signature verification is isolated in the HMAC Auth Service behind the ISignatureVerify interface. Shared provider secrets live only in that service, which is never exposed publicly, and the Gateway rejects any unverified payload before it reaches the broker. Internal services therefore only ever see authenticated events, and a compromise of a delivery or retry worker does not leak the signing keys. Only the Gateway sits on the public network, which keeps the attack surface to a single hardened entry point.

Performance Benefit

Asynchronous processing through the broker keeps ingestion latency low and constant: the Gateway only verifies, logs to the Event Store and publishes, then returns HTTP 2xx without waiting for downstream delivery. This avoids provider-side timeouts and duplicate resends. Horizontal scaling is targeted, so extra capacity is added only to the Forwarding Service or Retry Scheduler when backlog grows, instead of scaling the whole system.

Trade-offs Acknowledged

Microservices add operational overhead (broker management, monitoring, network hops) and require idempotent delivery since events may be retried. These are handled by storing event IDs and attempt counts in the Event Store, and are acceptable given the Hub's reliability and scaling needs.